

Aula 9 — Blazor Components Avançados

Módulo: Blazor

Carga horária: 4 horas

Projeto base: Aula08.BlazorServer

Objetivo da aula:

Dominar **Parameters**, **EventCallback**, **RenderFragments**, **Forms** e **Validation** no Blazor, construindo **formulários complexos**, **validação customizada** e um **dashboard interativo** com componentes desacoplados.

PARTE 1 — TEORIA (≈ 2 horas)

Parameters

◆ O que são?

Permitem passar dados do componente pai para o filho.

```
[Parameter] public string Titulo { get; set; }
```

Uso:

```
<Card Titulo="Produtos Ativos" />
```

- ✓ Comunicação unidirecional
 - ✓ Tipagem forte
 - ✓ Reutilização
-

EventCallback

◆ O que resolve?

Comunicação **filho** → **pai**, disparando eventos.

```
[Parameter] public EventCallback<int> OnExcluir { get; set; }
```

Uso:

```
await OnExcluir.InvokeAsync(id);
```

- ✓ Desacoplamento
 - ✓ Fluxo controlado
 - ✓ Padrão oficial Blazor
-

3 RenderFragments

◆ O que são?

Permitem **injetar conteúdo HTML/Blazor** dentro de componentes.

```
[Parameter] public RenderFragment ChildContent { get; set; }
```

Uso:

```
<Card> <p>Conteúdo dinâmico</p> </Card>
```

- ✓ Composição de UI
 - ✓ Layout flexível
 - ✓ Base de Design System
-

Forms e Validation

◆ EditForm

- Binding automático
- Integração com DataAnnotations

```
<EditForm Model="Produto" OnValidSubmit="Salvar">
```

◆ Validação customizada

- ValidationMessageStore
 - Regras além de DataAnnotations
-

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar:

- Componentes reutilizáveis com RenderFragments
 - Formulário complexo com validação customizada
 - Dashboard interativo com cards e métricas
-

Componente Base de Card (RenderFragment)

 Components/Card.razor

```
<div class="card"> <h4>@Titulo</h4> <div class="card-body"> @ChildContent </div> </div> @code { [Parameter] public string Titulo  
{ get; set; } [Parameter] public RenderFragment ChildContent { get; set; } }
```

Uso:

```
<Card Titulo="Resumo"> <p>Total de Produtos: @Total</p> </Card>
```

2 Formulário Complexo com Validação

Model com DataAnnotations

 Models/Produto.cs

```
using System.ComponentModel.DataAnnotations; public class Produto { public int Id { get; set; } [Required] [MinLength(3)] public string Nome { get; set; } [Range(0.01, 99999)] public decimal Preco { get; set; } public int Estoque { get; set; } }
```

Componente de Formulário Avançado

 Components/ProdutoFormAvancado.razor

```
@using Microsoft.AspNetCore.Components.Forms <EditForm EditContext="editContext" OnValidSubmit="Salvar">
<DataAnnotationsValidator /> <ValidationSummary /> <InputText @bind-Value="Produto.Nome" placeholder="Nome" /> <InputNumber
@bind-Value="Produto.Preco" placeholder="Preço" /> <InputNumber @bind-Value="Produto.Estoque" placeholder="Estoque" /> <button
type="submit">Salvar</button> </EditForm> @code { [Parameter] public Produto Produto { get; set; } = new(); [Parameter] public
EventCallback<Produto> OnSalvar { get; set; } private EditContext editContext; private ValidationMessageStore messages; protected
override void OnInitialized() { editContext = new EditContext(Produto); messages = new ValidationMessageStore(editContext);
editContext.OnValidationRequested += CustomValidation; } void CustomValidation(object sender, ValidationRequestedEventArgs e) {
messages.Clear(); if (Produto.Estoque < 0) { messages.Add( () => Produto.Estoque, "Estoque não pode ser negativo"); } } async
Task Salvar() { await OnSalvar.InvokeAsync(Produto); } }
```

- ✓ DataAnnotations + regras customizadas
- ✓ EditContext manual

✓ Validação profissional

3 Dashboard Interativo

Página Dashboard

 Pages/Dashboard.razor

```
@page "/dashboard" @inject ProdutoService ProdutoService <h3>Dashboard</h3> <div class="dashboard"> <Card Titulo="Total de
Produtos"> <h2>@TotalProdutos</h2> </Card> <Card Titulo="Valor Médio"> <h2>@PrecoMedio.ToString("C")</h2> </Card> </div> @code {
int TotalProdutos; decimal PrecoMedio; protected override void OnInitialized() { var produtos = ProdutoService.ObterTodos();
TotalProdutos = produtos.Count; PrecoMedio = produtos.Any() ? produtos.Average(p => p.Preco) : 0; } }
```

- ✓ Cards reutilizáveis
- ✓ Métricas em tempo real
- ✓ UI orientada a dados

4 Comunicação entre Componentes (Resumo)

Fluxo	Técnica
Pai → Filho	[Parameter]
Filho → Pai	EventCallback
Conteúdo	RenderFragment
Estado	Serviço Scoped

✓ Resultado Esperado

- ✓ Componentes altamente reutilizáveis
 - ✓ Validação customizada
 - ✓ Formulários complexos
 - ✓ Dashboard interativo
 - ✓ Base sólida para Blazor WASM
-

📌 Atividade Avaliativa

👉 Desafio prático:

1. Criar `Card` com ícone dinâmico
2. Criar validação de preço mínimo por categoria
3. Atualizar dashboard em tempo real após cadastro
4. Usar `CascadingParameter` para tema (dark/light)